arXiv:1804.10332v2 [cs.RO] 16 May 2018

# Sim-to-Real: Learning Agile Locomotion For Quadruped Robots

Jie Tan[1], Tingnan Zhang[1], Erwin Coumans[1], Atil Iscen[1],
Yunfei Bai[2], Danijar Hafner[1], Steven Bohez[3], and Vincent Vanhoucke[1]

[1]Google Brain
[2]X
[3]Google DeepMind

***Abstract*—Designing agile locomotion for quadruped robots often requires extensive expertise and tedious manual tuning. In this paper, we present a system to automate this process by leveraging deep reinforcement learning techniques. Our system can learn quadruped locomotion from scratch using simple reward signals. In addition, users can provide an open loop reference to guide the learning process when more control over the learned gait is needed. The control policies are learned in a physics simulator and then deployed on real robots. In robotics, policies trained in simulation often do not transfer to the real world. We narrow this reality gap by improving the physics simulator and learning robust policies. We improve the simulation using system identification, developing an accurate actuator model and simulating latency. We learn robust controllers by randomizing the physical environments, adding perturbations and designing a compact observation space. We evaluate our system on two agile locomotion gaits: trotting and galloping. After learning in simulation, a quadruped robot can successfully perform both gaits in the real world.**

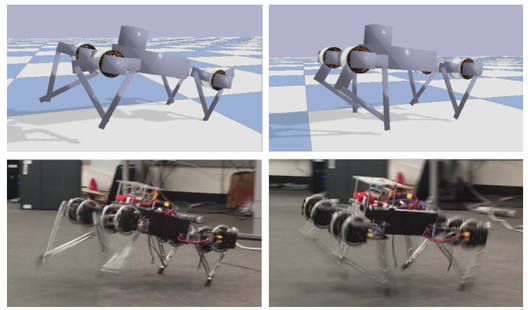


Fig. 1: The simulated and the real Minitaurs learned to gallop using deep reinforcement learning.

## I. Introduction

Designing agile locomotion for quadruped robots is a long-standing research problem [1]. This is because it is difficult to control an under-actuated robot performing highly dynamic motion that involve intricate balance. Classical approaches often require extensive experience and tedious manual tuning [2, 3]. Can we automate this process?

Recently, we have seen tremendous progress in deep reinforcement learning (deep RL) [4, 5, 6]. These algorithms can solve locomotion problems from scratch without much human intervention. However, most of these studies are conducted in simulation, and a controller learned in simulation often performs poorly in the real world. This *reality gap* [7, 8] is caused by model discrepancies between the simulated and the real physical system. Many factors, including unmodeled dynamics, wrong simulation parameters, and numerical errors, contribute to this gap. Even worse, this gap is greatly amplified in locomotion tasks. When a robot performs agile motion with frequent contact changes, the switches of contact situations break the control space into fragmented pieces. Any small model discrepancy can be magnified and generate bifurcated consequences. Overcoming the reality gap is challenging.

An alternative is to learn the task directly on the physical system. While this has been successfully demonstrated in robotic grasping [9], it is challenging to apply this method to locomotion tasks due to the difficulties of automatically resetting the experiments and continuously collecting data. In addition, every falling during learning can potentially damage the robot. Thus for locomotion tasks, learning in simulation is more appealing because it is faster, cheaper and safer.

In this paper, we present a complete learning system for agile locomotion, in which control policies are learned in simulation and deployed on real robots. There are two main challenges: 1) learning *controllable* locomotion policies; and 2) transferring the policies to the physical system.

While learning from scratch can lead to better policies than incorporating human guidance [10], in robotics, having control of the learned policy sometimes is preferred. Our learning system provides users a full spectrum of controllability over the learned policies. The user can choose from letting the system learn completely by itself to specifying an open-loop reference gait as human guidance. Our system will keep the learned gait close to the reference while, at the same time, maintain balance, increase speed and energy efficiency.

To narrow the reality gap, we perform system identification to find the correct simulation parameters. Besides, we improve the fidelity of the physics simulator by adding a faithful actuator model and latency handling. To further narrow the gap, we experiment with three approaches to increase the robustness of the learned controllers: dynamics randomization,

arXiv:1804.10332v2 [cs.RO] 16 May 2018

# 从仿真到现实：四足机器人敏捷运动学习

Jie Tan[1], 、Tingnan Zhang[1]、、Erwin Coumans[1]、、Atil Iscen[1]、、
Yunfei Bai[2], 、Danijar Hafner[1]、、Steven Bohez[3],和 Vincent Vanhoucke[1]

[1]谷歌大脑 [2]X [3]
Google
DeepMind

摘要——为四足机器人设计敏捷运动通常需要广泛的专业知识和繁琐的手动调节。本文提出了一个利用深度强化学习技术自动化该过程的系统。我们的系统仅使用简单的奖励信号，即可从零开始学习四足运动。此外，当需要对学习到的步态进行更多控制时，用户可以提供开环参考来引导学习过程。控制策略在物理仿真器中训练，然后部署到真实机器人上。在机器人领域，仿真中训练的策略往往无法迁移到现实世界。我们通过改进物理仿真器和学习鲁棒策略来缩小这一现实差距。我们利用系统辨识改进仿真，开发了精确的执行器模型并模拟了延迟。我们通过随机化物理环境、添加扰动以及设计紧凑的观测空间来学习鲁棒控制器。我们在两种敏捷运动步态上评估了系统：小跑和飞奔。在仿真中学习后，四足机器人能够在现实世界中成功执行这两种步态。

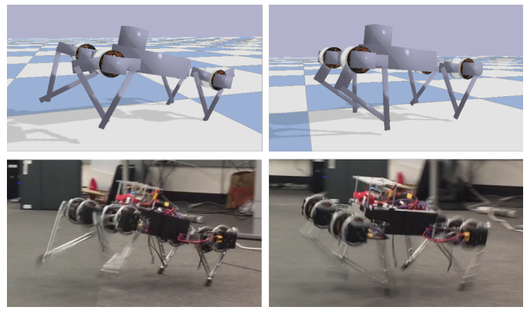


图1：仿真和真实的Minitaurs通过深度强化学习学会了疾驰。

## I. 引言

为四足机器人设计敏捷运动是一个长期存在的研究问题 [1]。这是因为控制一个欠驱动机器人执行涉及复杂平衡的高度动态运动非常困难。经典方法通常需要丰富的经验和繁琐的手动调参[2, 3]。我们能否将这一过程自动化?

最近，我们在深度强化学习（deep RL）方面取得了巨大进展 [4, 5, 6]。这些算法可以几乎无需人工干预地从零开始解决运动控制问题。然而，这些研究大多在仿真中进行，在仿真中学习的控制器在现实世界中往往表现不佳。这种现实差距 [7, 8]是由仿真系统与真实物理系统之间的模型差异造成的。造成这种差距的因素有很多，包括未建模的动力学、错误的仿真参数和数值误差。更糟的是，这种差距在运动任务中会被大幅放大。当机器人执行频繁接触变化的敏捷运动时，接触状态的切换会将控制空间分割成碎片。任何微小的模型差异都可能被放大，并产生分叉式的后果。克服现实差距充满挑战。

另一种方法是直接在物理系统上学习该任务。虽然这种方法已在机器人抓取 [9]，中得到成功验证，但将该方法应用于运动任务时，由于自动重置实验和持续收集数据存在困难，因此颇具挑战。此外，学习过程中的每一次摔倒都可能损坏机器人。因此，对于运动任务而言，在仿真中学习更具吸引力，因为这样更快、更便宜、更安全。

在本文中，我们提出了一个完整的敏捷运动学习系统，其中控制策略在仿真中学习并部署到真实机器人上。主要存在两个挑战：1）学习可控的运动策略；2）将策略迁移到物理系统。

虽然从零开始学习相较于在机器人技术中加入人工引导 [10], 可以产生更好的策略，但有时对学习到的策略进行控制更为可取。我们的学习系统为用户提供了对学习策略的全方位可控性。用户既可以让系统完全自行学习，也可以指定一个开环参考步态作为人工引导。我们的系统将使学习到的步态接近参考步态，同时保持平衡、提高速度和能效。

为了缩小现实差距，我们执行系统辨识以找到正确的仿真参数。此外，我们通过添加精确的执行器模型和延迟处理来提高物理仿真器的保真度。为了进一步缩小差距，我们尝试了三种方法来增强学习到的控制器的鲁棒性：动力学随机化、

perturbation forces, and compact design of observation space.

We evaluate our system on a quadruped robot with two locomotion tasks, trotting and galloping, in Section VI. We show that with deep RL, highly agile locomotion gaits can emerge automatically. We also demonstrate how users can easily specify the style of locomotion using our system. When comparing with the gaits handcrafted by experts, we find that our learned gaits are more energy efficient at the same running speed. We demonstrate that with an accurate physics simulator and robust control policies, we can successfully deploy policies learned in simulation to the physical system. The main contributions of this paper are:

1) We propose a complete learning system for agile locomotion. It provides users a full spectrum (from fully restricted to a user-specified gait to fully learned from scratch) of controllability over the learned policies.
2) We show that the reality gap can be narrowed by a variety of approaches and conduct comprehensive evaluations on their effectiveness.
3) We demonstrate that agile locomotion gaits, such as trotting and galloping, can be learned automatically and these gaits can work on robots directly without further training on the physical system.

## II. Related Work

### A. Legged Locomotion Control

Optimizing controllers [11] automatically is an appealing alternative to tedious manual tuning. Popular optimization methods for locomotion control include black-box [12] and Bayesian optimization [13]. Bayesian optimization is often data efficient enough to be applied directly on real robots [14, 15]. However, it is challenging to scale these methods to high-dimensional control space. For this reason, feature engineering and controller architecture design are needed.

On the other hand, recent advances in deep RL have significantly reduced the requirement of human expertise [6]. We have witnessed intense competitions among deep RL algorithms in the simulated benchmark environments [16, 17]. In this paper, we choose to use Proximal Policy Optimization (PPO) [5] because it is a stable on-policy method and can be easily parallelized [18].

Much research has applied reinforcement learning to locomotion tasks [19, 20, 21, 22, 23]. More recently, Gay et al. [24] learned a neural network that modified a Central Pattern Generator controller for stable quadruped locomotion. Levine et al. [25] applied guided policy search to learn bipedal locomotion in simulation. Peng et al. [26] developed a CACLA-inspired algorithm to control a simulated dog to navigate complex 1D terrains. It was further improved using a mixture of actor-critic experts [27]. Peng et al. [28] learned a hierarchical controller to direct a 3D biped to walk in a simulated environment. Heess et al. [29] showed that complex locomotion behaviors, such as running, jumping and crouching, can be learned directly from simple reward signals in rich simulated environments. Sharma and Kitani [30] exploited the cyclic nature of locomotion with phase-parametric policies. Note that in most of these latest work, learning and evaluation were exclusively performed in simulation. It is not clear whether these learned policies can be safely and successfully deployed on the robots. In this paper, while we also train in simulation, more importantly, we test the policies on the real robot and explore different approaches to narrow the reality gap.

### B. Overcoming the Reality Gap

Reality gap is the major obstacle to applying deep RL in robotics. Neunert et al. [31] analyzed potential causes of the reality gap, some of which can be solved by system identification [32]. Li et al. [33] showed that the transferability of open loop locomotion could be increased if carefully measured physical parameters were used in simulation. These physical parameters can also be optimized by matching the robot behaviors in the simulated and the real world [34, 35]. Bongard et al. [36] used the actuation-sensation relationship to build and refine a simulation through continuous self-modeling, and later used this model to plan forward locomotion. Ha et al. [37] used Gaussian Processes, a non-parametric model, to minimize the error between simulation and real physics. Yu et al. [38] performed online system identification explicitly on physical parameters, while Peng et al. [39] embedded system identification implicitly into a recurrent policy. In addition to system identification, we identify that inaccurate actuator models and lack of latency modeling are two major causes of the reality gap. We improve our simulator with new models.

A robust controller is more likely to be transferred to the real world. Robustness can be improved by injecting noise [40], perturbing the simulated robot [41], leveraging multiple simulators [8], using domain randomization [42] and dynamics randomization [43, 44, 39]. Although not explicitly using real-world data for training, these methods have been shown effective to increase the success rate of sim-to-real transfer.

Another way to cross the reality gap is to learn from both simulation and real-world data. A policy can be pre-trained in simulation and then fine-tuned on robots [45]. Hanna and Stone [46] adapted the policy using a learned action space transformation from simulation to the real world. In visuomotor learning, domain adaptation were applied at feature level [47, 48] or pixel level [49] to transfer the controller. Bousmalis et al. [49] reduced the real world data requirement by training a generator network that converts simulated images to real images. While some of these methods were successfully demonstrated in robotic grasping, it is challenging to apply them to locomotion due to the difficulties to continuously and safely collect enough real world data. Furthermore, we need to narrow the reality gap of dynamics rather than perception.

## III. Robot Platform and Physics Simulation

Our robot platform is the Minitaur from Ghost Robotics (Figure 1 bottom), a quadruped robot with eight direct-drive actuators [50]. Each leg is controlled by two actuators that allow it to move in the sagittal plane. The motors can be actuated through position control or through a Pulse Width

扰动力，以及紧凑的观测空间设计。

我们在第六节中，在四足机器人上评估了我们的系统，包含两种运动任务：小跑和疾驰。我们证明，借助深度强化学习，高度敏捷的运动步态可以自动涌现。我们还展示了用户如何通过我们的系统轻松指定运动风格。与专家手工设计的步态相比，我们发现学习得到的步态在相同奔跑速度下能效更高。我们证明，凭借精确的物理模拟器和鲁棒的控制策略，我们可以成功地将仿真中学习到的策略部署到物理系统上。本文的主要贡献如下：

1) 我们提出了一套完整的敏捷运动学习系统。它为用户提供了对学习策略的全面可控性（从完全受限到用户指定的步态，再到完全从头学习）。2) 我们表明，现实差距可以通过多种方法缩小，并对这些方法的有效性进行了全面评估。3) 我们证明，诸如小跑和疾驰等敏捷运动步态可以自动学习，并且这些步态可以直接在机器人上工作，无需在物理系统上进行进一步训练。

## II. 相关工作

### A. 足式运动控制

自动优化控制器 [11] 是替代繁琐手动调参的一种有吸引力的选择。足式运动控制的常用优化方法包括黑盒 [12]和贝叶斯优化 [13]。贝叶斯优化通常数据效率足够高，可以直接应用于真实机器人[14, 15]。然而，将这些方法扩展到高维控制空间颇具挑战。因此，需要进行特征工程和控制器架构设计。

另一方面，深度强化学习的最新进展显著降低了对人类专业知识的需求 [6]。我们见证了深度强化学习算法在模拟基准环境中的激烈竞争 [16, 17]。在本文中，我们选择使用近端策略优化（PPO） [5]，因为它是一种稳定的同策略方法，并且易于并行化 [18]。

许多研究已将强化学习应用于运动任务 [19, 20, 21, 22, 23]。最近，Gay等人 [24]学习了一个神经网络，修改了中央模式生成器控制器，以实现稳定的四足运动。Levine等人[25] 应用引导策略搜索在仿真中学习双足运动。Peng等人 [26] 开发了一种受CACLA启发的算法，以控制模拟狗在复杂的一维地形中导航。该算法随后使用混合的演员-评论家专家 [27]得到进一步改进。Peng等人 [28]学习了一个分层控制器，用于在模拟环境中引导三维双足机器人行走。Heess等人 [29] 表明，诸如跑步、跳跃和下蹲等复杂运动行为可以在丰富的模拟环境中直接从简单的奖励信号中学习。Sharma和Kitani [30] 利用了运动的周期性，

相位参数化策略。注意，在最近的大多数此类工作中，学习和评估完全在仿真中进行。目前尚不清楚这些学习到的策略能否安全且成功地部署到机器人上。在本文中，我们也进行仿真训练，但更重要的是，我们在真实机器人上测试这些策略，并探索不同方法来缩小现实差距。

### B. 克服现实差距

现实差距是将深度强化学习应用于机器人领域的主要障碍。Neunert等人 [31] 分析了现实差距的潜在成因，其中一些可以通过系统辨识来解决 [32]。Li等人 [33]表明，如果在仿真中使用经过仔细测量的物理参数，开环运动的可迁移性可以得到提高。这些物理参数还可以通过匹配仿真与真实世界中的机器人行为进行优化 [34, 35]。Bongard等人 [36] 利用驱动-感知关系，通过连续自建模构建并完善仿真模型，随后使用该模型规划前向运动。Ha等人[37]使用高斯过程这一非参数模型来最小化仿真与真实物理之间的误差。Yu等人 [38] 对物理参数进行显式的在线系统辨识，而Peng等人 [39] 则将系统辨识隐式地嵌入到循环策略中。除了系统辨识之外，我们还发现执行器模型不准确和缺乏延迟建模是现实差距的两个主要原因。我们使用新模型对仿真器进行了改进。

鲁棒控制器更有可能迁移到真实世界。可以通过注入噪声[40]、扰动仿真机器人 [41]、利用多个仿真器 [8]、使用域随机化 [42]以及动力学随机化 [43, 44, 39]来提高鲁棒性。虽然这些方法没有明确使用真实世界数据进行训练，但已被证明能有效提高仿真到现实迁移的成功率。

跨越现实差距的另一种方法是同时从仿真和真实世界数据中学习。策略可以在仿真中预训练，然后在机器人 [45]上进行微调。Hanna 和 Stone [46] 利用从仿真到真实世界学习到的动作空间变换来调整策略。在视觉-运动学习中，域适应被应用于特征层面 [47, 48] 或像素层面 [49] ，以迁移控制器。Bousmalis 等人 [49]通过训练一个将仿真图像转换为真实图像的生成器网络，降低了对真实世界数据的需求。虽然其中一些方法在机器人抓取中得到了成功验证，但由于难以持续且安全地收集足够的真实世界数据，将这些方法应用于运动控制仍具挑战性。此外，我们需要缩小的是动力学方面的现实差距，而非感知方面的现实差距。

## III. 机器人平台与物理仿真

我们的机器人平台是来自 Ghost Robotics 的 Minitaur（图1底部），这是一款具有八个直驱执行器的四足机器人 [50]。每条腿由两个执行器控制，使其能够在矢状面上运动。电机既可以通过位置控制来驱动，也可以通过脉冲宽度

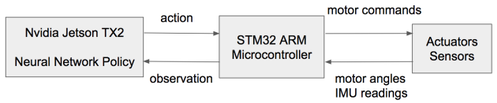



Fig. 2: The customized hardware architecture enables the Minitaur to perform deep neural network inference.

Modulation (PWM) signal. The Minitaur is equipped with motor encoders that measure the motor angles and an IMU that measures the orientation and the angular velocity of its base. An STM32 ARM microcontroller sends commands to actuators, receives sensor readings and can perform simple computations. However, this microcontroller is not powerful enough to execute neural network policies learned from deep RL. For this reason, we installed an Nvidia Jetson TX2 to perform neural network inference. The TX2 is interfaced with the microcontroller through UART communication. At every control step, the sensor measurements are collected at the microcontroller and sent back to the TX2, where they are fed into a neural network policy to decide the actions to take. These actions are then transmitted to the microcontroller and executed by the actuators (Figure 2). Since the TX2 does not run a real time operating system, the control loop runs at variable control frequencies of approximately 150-200Hz.

We build a physics simulation of the Minitaur (Figure 1 top) using PyBullet [51], a Python module that extends the Bullet Physics Engine with robotics and machine learning capabilities. Bullet solves the equations of motion for articulated rigid bodies in generalized coordinates while simultaneously satisfying physical constraints including contact, joint limits and actuator models. Then the state of the system is numerically integrated over time using a semi-implicit scheme.

## IV. Learning Locomotion Controllers

### A. Background

We formulate locomotion control as a Partially Observable Markov Decision Process (POMDP) and solve it using a policy gradient method. An MDP is a tuple $(S, A, r, D, P_{sas'}, \gamma)$, where $S$ is the state space; $A$ is the action space; $r$ is the reward function; $D$ is the distribution of initial states $s_0$, $P_{sas'}$ is the transition probability; and $\gamma \in [0, 1]$ is the discount factor. Our problem is partially observable because certain states such as the position of the Minitaur's base and the foot contact forces are not accessible due to lack of corresponding sensors. At every control step, a partial observation $o \in O$, rather than a complete state $s \in S$, is observed. Reinforcement learning optimizes a policy $\pi : O \mapsto A$ that maximizes the expected return (accumulated rewards) $R$.

$$\pi^* = arg\,max_\pi E_{s_0 \sim D}[R_\pi(s_0)] \tag{1}$$

### B. Observation and Action Space

In our problem, the observations include the roll, pitch, and the angular velocities of the base along these two axes, and the eight motor angles. Note that we do not include all available sensor measurements in the observation space. For example, the IMU also provides the yaw of the base. We exclude it because the measurement drifts quickly. The motor velocities can also be calculated but can be noisy. We find that our observation space is sufficient to learn the tasks demonstrated in this paper. More importantly, a compact observation space helps to transfer the policy to the real robot. More analysis on this is presented in Section VI.

When designing the action space, we choose to use the position control mode of the actuators for safety and ease of learning [52]. The actions consist of the desired pose of each leg in the *leg space* [50, 12]. The pose of each leg is decomposed into the swing and the extension components $(s, e)$ (Figure 3). They are mapped into the *motor space* as

$$\begin{aligned} \theta_1 &= e + s \\ \theta_2 &= e - s \end{aligned}$$

where $\theta_1$ and $\theta_2$ are the angles of the two motors controlling the same leg; $s$ and $e$ are the swing and the extension components in the leg space.

An alternative action space is the eight desired motor angles. However, in this motor space, many configurations are invalid due to self collisions between body parts. This results in an action space where valid actions are scattered nonconvex regions, which significantly increases the difficulty of learning. In contrast, in the leg space, we can easily set a rectangle bound that prunes out all the invalid actions while still covering most of the valid configurations.

### C. Reward Function

We design a reward function to encourage faster forward running speed and penalize high energy consumption.

$$r = (\mathbf{p}_n - \mathbf{p}_{n-1}) \cdot \mathbf{d} - w\Delta t|\boldsymbol{\tau}_n \cdot \dot{\mathbf{q}}_n| \tag{2}$$

where $\mathbf{p}_n$ and $\mathbf{p}_{n-1}$ are the positions of the Minitaur's base at the current and the previous time step respectively; $\mathbf{d}$ is the desired running direction; $\Delta t$ is the time step; $\boldsymbol{\tau}$ are the motor torques and $\dot{\mathbf{q}}$ are the motor velocities. The first term measures the running distance towards the desired direction and the second term measures the energy expenditure. $w$ is the weight that balances these two terms. Since the learning algorithm is robust to a wide range of $w$, we do not tune it and use

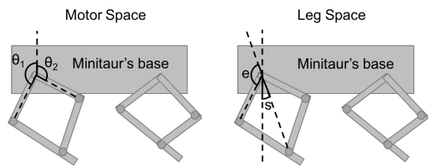



Fig. 3: Representation of the leg pose in motor space and leg space. Extension (e) sets the length of the leg by rotating both motors in opposite directions while swing (s) sets the overall rotation of the leg by rotating both motors in same direction.

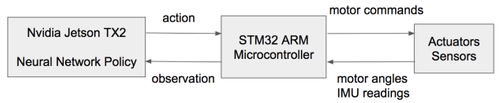



图2：定制硬件架构使Minitaur能够执行深度神经网络推理。

调制（PWM）信号。Minitaur 配备了用于测量电机角度的电机编码器，以及用于测量其基座姿态和角速度的 IMU。STM32 ARM 微控制器向执行器发送命令、接收传感器读数，并能执行简单计算。然而，该微控制器的性能不足以执行通过深度强化学习学到的神经网络策略。因此，我们安装了 Nvidia Jetson TX2 来执行神经网络推理。TX2 通过 UART 通信与微控制器连接。在每个控制周期，微控制器收集传感器测量值并将其发送回 TX2，由 TX2 将这些测量值输入神经网络策略，以决定要执行的动作。随后这些动作被传输到微控制器，并由执行器执行（图 2）。由于 TX2 不运行实时操作系统，控制回路以约 150-200Hz 的可变控制频率运行。

我们使用PyBullet [51], 构建Minitaur（图1顶部）的物理仿真，PyBullet是一个Python模块，通过机器人和机器学习功能扩展了Bullet物理引擎。Bullet在广义坐标中求解铰接刚体的运动方程，同时满足包括接触、关节限制和执行器模型在内的物理约束。然后，使用半隐式格式对系统状态进行随时间数值积分。

## IV. 学习运动控制器

### A. 背景

我们将运动控制建模为部分可观测马尔可夫决策过程（POMDP），并使用策略梯度方法求解。MDP是一个元组 $(S, A, r, D, P_{sas'}, \gamma)$，其中 $S$ 是状态空间； $A$ 是动作空间； $r$ 是奖励函数； $D$ 是初始状态 $s_0$ 的分布； $P_{sas'}$ 是转移概率； $\gamma \in [0, 1]$ 是折扣因子。我们的问题具有部分可观测性，因为某些状态（如Minitaur基座的位置和足底接触力）由于缺少相应传感器而无法获取。在每个控制步中，观测到的是部分观测 $o \in O$，而非完整状态 $s \in S$。强化学习优化策略 $\pi : O \mapsto A$，使期望回报（累积奖励） $R$ 最大化。

$$\pi^* = arg\,max_\pi E_{s_0 \sim D}[R_\pi(s_0)] \tag{1}$$

### B. 观测与动作空间

在我们的问题中，观测包括横滚角、俯仰角、机体沿这两个轴的角速度，以及八个电机角度。注意，我们并没有将所有可用的传感器测量都包含在观测空间中。例如，IMU还提供机体的偏航角。我们将其排除在外，因为该测量值会快速漂移。电机速度也可以计算出来，但可能带有噪声。我们发现，我们的观测空间足以学习本文所演示的任务。更重要的是，紧凑的观测空间有助于将策略迁移到真实机器人上。关于这一点的更多分析见第六节。

在设计动作空间时，为了安全且易于学习，我们选择使用执行器的位置控制模式 [52]。动作由每条腿在腿部空间中的期望位姿组成 [50, 12]。每条腿的位姿被分解为摆动分量和伸展分量（$s$, $e$）（图3）。它们被映射到电机空间，如下所示：

$$\begin{aligned} \theta_1 &= e + s \\ \theta_2 &= e - s \end{aligned}$$

其中 $\theta_1$ 和 $\theta_2$ 是控制同一条腿的两个电机的角度； $s$ 和 $e$ 是腿部空间中的摆动分量和伸展分量。

另一种动作空间是八个期望电机角度。然而，在这种电机空间中，由于身体部位之间的自碰撞，许多配置是无效的。这导致动作空间中的有效动作分散在非凸区域中，从而显著增加了学习难度。相比之下，在腿部空间中，我们可以轻松设置一个矩形边界，在剪除所有无效动作的同时，仍覆盖大多数有效配置。

### C. 奖励函数

我们设计了一个奖励函数，以鼓励更快的向前奔跑速度，并对高能耗进行惩罚。

$$r = (\mathbf{p}_n - \mathbf{p}_{n-1}) \cdot \mathbf{d} - w\Delta t|\boldsymbol{\tau}_n \cdot \dot{\mathbf{q}}_n| \tag{2}$$

其中 $\mathrm{p}_n$ 和 $\mathrm{p}_{n-1}$ 分别是 Minitaur 机体在当前时间步和上一时间步的位置； d 是期望的奔跑方向； $\Delta t$ 是时间步； $\boldsymbol{\tau}$ 是电机扭矩， $\dot{}$q 是电机速度。第一项衡量沿期望方向奔跑的距离，第二项衡量能量消耗。 $w$ 是平衡这两项的权重。由于学习算法对较宽的 $w$ 范围具有鲁棒性，我们不调节它，而是使用

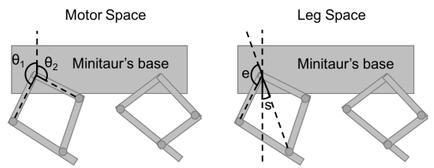



图3：腿部姿态在马达空间和腿部空间中的表示。伸展（e）通过让两个马达反向旋转来设置腿的长度，而摆动（s）通过让两个马达同向旋转来设置腿的整体旋转角度。

$w = 0.008$ in all our experiments. During training, the rewards are accumulated at each episode. An episode terminates after 1000 steps or when the simulated Minitaur loses balance: its base tilts more than 0.5 radians.

### D. Policy Representation

Although learning from scratch can eliminate the need of human expertise, and sometimes achieve better performance, having control of the learned policies is important for robotic applications. For example, we may want to specify details of a gait (e.g. style or ground clearance). For this reason, we decouple the locomotion controller into two parts, an open loop component that allows a user to provide reference trajectories and a feedback component that adjusts the leg poses on top of the reference based on the observations.

$$\mathbf{a}(t, \mathbf{o}) = \bar{\mathbf{a}}(t) + \pi(\mathbf{o}) \tag{3}$$

where $\bar{\mathbf{a}}(t)$ is the open loop component, which is typically a periodic signal, and $\pi(\mathbf{o})$ is the feedback component. In this way, users can easily express the desired gait using an open loop signal and learning will figure out the rest, such as the balance control, which is tedious to design manually.

This hybrid policy (eq. (3)) is a general formulation that gives users a full spectrum of controllability. It can be varied continuously from fully user-specified to entirely learned from scratch. If we want to use a user-specified policy, we can set both the lower and the upper bounds of $\pi(\mathbf{o})$ to be zero. If we want a policy that is learned from scratch, we can set $\bar{\mathbf{a}}(t) = 0$ and give the feedback component $\pi(\mathbf{o})$ a wide output range. By varying the open loop signal and the output bound of the feedback component, we can decide how much user control is applied to the system. In Section VI, we will illustrate two examples, learning to gallop from scratch and learning to trot with a user provided reference.

We represent the feedback component $\pi$ with a neural network and solve the above POMDP using Proximal Policy Optimization [5]. The neural network has two fully-connected hidden layers. Its size is determined via hyperparameter search. Refer to Section VI for more details.

## V. Narrowing the Reality Gap

Due to the reality gap, robotic controllers learned in simulation usually do not perform well in the real environments. We propose two approaches to narrow the gap: improving simulation fidelity and learning robust controllers.

### A. Improving Simulation Fidelity

Since the reality gap is caused by model discrepancies between the simulation and the real dynamics, a direct way to narrow it is to improve the simulation. We first create an accurate Unified Robot Description Format (URDF) [53] file for the simulated Minitaur. We disassemble a Minitaur[1], measure the dimension, weigh the mass, find the center of mass of each link and incorporate this information into the URDF file. Measuring inertia is difficult. Instead, we estimate it for each link given its shape and mass, assuming uniform density. We also design experiments to measure motor frictions [50]. In addition to system identification, we augment the simulator with a more faithful actuator model and latency handling.

*a) Actuator Model:* We use position control to actuate the motors of the Minitaur. Bullet also provides position control for simulated motors. In its implementation, one constraint $e_{n+1} = 0$ is formulated for each motor where $e_{n+1}$ is an error at the end of current time step. The error is defined as

$$e_{n+1} = k_p(\bar{q} - q_{n+1}) + k_d(\bar{\dot{q}} - \dot{q}_{n+1}) \tag{4}$$

where $\bar{q}$ and $\bar{\dot{q}}$ are the desired motor angle and velocity, $q_{n+1}$ and $\dot{q}_{n+1}$ are the motor angle and velocity at the end of current time step, $k_p$ is the proportional gain and $k_d$ is the derivative gain. Despite its similarity to the Proportional-Derivative (PD) servo, a key difference is that eq. (4) guarantees that the motor angle and velocity at the end of the time step satisfy the constraint while PD servo uses the current motor angle and velocity to decide how to actuate the motors. As a result, if large gains are used, the motors can remain stable in simulation but oscillate in reality.

To eliminate the model discrepancy for actuators, we develop an actuator model according to the dynamics of an ideal DC motor. Given a PWM signal, the torque of the motor is

$$\begin{aligned} \tau &= K_t I \\ I &= \frac{V_{\text{pwm}} - V_{\text{emf}}}{R} \qquad (5) \\ V_{\text{emf}} &= K_t \dot{q} \qquad (6) \end{aligned}$$

where $I$ is the armature current, $K_t$ is the torque constant or back electromotive force (EMF) constant, $V_{\text{pwm}}$ is the supplied voltage which is modulated by the PWM signal, $V_{\text{emf}}$ is the back EMF voltage, and $R$ is the armature resistance. The parameters $K_t$ and $R$ are provided in the actuator specification.

Using the above model, we observe that the real Minitaur often sinks to its feet or cannot lift them while the same controller works fine in simulation. This is because the linear torque-current relation only holds for ideal motors. In reality, the torque saturates as the current increases. For this reason, we construct a piece-wise linear function to characterize this nonlinear torque-current relation [54]. In simulation, once the current is computed from PWM (eq. (5) and (6)), we use this piece-wise function to look up the corresponding torque.

In position control, PWM is controlled by a PD servo.

$$V_{\text{pwm}} = V(k_p(\bar{q} - q_n) + k_d(\bar{\dot{q}} - \dot{q}_n)) \tag{7}$$

where $V$ is the battery voltage. Note that we use the angle and velocity at the current time step. In addition, we set the target velocity $\bar{\dot{q}} = 0$ after consulting Ghost Robotics's PD implementation on the microcontroller.

We performed an experiment to validate the new actuator model. We actuated a motor with a desired trajectory of sine curve. With this actuator model, the simulated trajectory agrees with the ground truth (Figure 4).

[1]For robots that are difficult to dissemble, traditional system identification methods could be applied.

$w = 0.008$ 在我们所有的实验中。训练期间，奖励在每个回合中累积。一个回合在 1000 步之后或当模拟的 Minitaur 失去平衡时终止：其机体倾斜超过 0.5 弧度。

### D. 策略表示

尽管从头学习可以消除对人类专业知识的需求，有时还能取得更好的性能，但掌控所习得的策略对于机器人应用非常重要。例如，我们可能希望指定步态的细节（如风格或离地间隙）。出于这一原因，我们将运动控制器解耦为两个部分：一个开环组件，允许用户提供参考轨迹；一个反馈组件，根据观测在参考轨迹之上调整腿部姿态。

$$\mathbf{a}(t, \mathbf{o}) = \bar{\mathbf{a}}(t) + \pi(\mathbf{o}) \tag{3}$$

其中 $\bar{\mathbf{a}}(t)$ 是开环组件，通常是一个周期信号，而 $\pi(\mathbf{o})$ 是反馈组件。这样，用户可以轻松地用开环信号表达所需步态，而学习算法将负责其余部分，例如手动设计起来很繁琐的平衡控制。

这种混合策略（式 (3)）是一种通用表述，为用户提供全面的可控性。它可以在完全由用户指定到完全从头学习之间连续变化。如果我们想使用用户指定的策略，可以将 $\pi(\mathbf{o})$ 的下界和上界都设为零。如果我们想要一个从头学习的策略，可以设置 $\bar{\mathbf{a}}(t) = 0$，并给反馈组件 $\pi(\mathbf{o})$ 一个较宽的输出范围。通过改变开环信号和反馈组件的输出范围，我们可以决定对系统施加多少用户控制。在第六节中，我们将说明两个示例：从零开始学习疾驰，以及使用用户提供的参考学习小跑。

我们使用神经网络表示反馈组件 $\pi$，并使用近端策略优化 [5]求解上述 POMDP。该神经网络具有两个全连接隐藏层。其规模通过超参数搜索确定。更多细节请参阅第 VI 节。

## V. 缩小现实差距

由于现实差距的存在，在仿真中训练得到的机器人控制器通常在真实环境中表现不佳。我们提出了两种缩小该差距的方法：提高仿真保真度和学习鲁棒控制器。

### A. 提高仿真保真度

由于现实差距源于仿真与真实动力学之间的模型差异，缩小这一差距的直接方法就是改进仿真。我们首先为仿真的 Minitaur 创建了一个精确的统一机器人描述格式（URDF）[53]文件。我们拆解了一台 Minitaur[1]，测量尺寸、称量质量、找出每个连杆的质心，并将这些信息纳入到 URDF 文件中。测量惯量很困难。因此，我们根据每个连杆的形状和质量，假设密度均匀，估算其惯量。我们还设计了实验来测量电机摩擦力[50]。除了系统辨识外，我们还为仿真器增加了更真实的执行器模型和延迟处理。

*a)* 执行器模型：我们使用位置控制来驱动 Minitaur 的电机。Bullet 也为仿真电机提供了位置控制。在其实现中，为每个电机建立一个约束$e_{n+1} = 0$，其中 $e_{n+1}$ 是当前时间步结束时的误差。该误差定义为

$$e_{n+1} = k_p(\bar{q} - q_{n+1}) + k_d(\bar{\dot{q}} - \dot{q}_{n+1}) \tag{4}$$

其中 $\bar{q}$ 和 $\dot{q}$ 分别是期望的电机角度和速度，$q_{n+1}$ 和 $\dot{q}_{n+1}$ 是当前时间步结束时电机的角度和速度，$k_p$ 是比例增益，$k_d$ 是微分增益。尽管它与比例-微分（PD）伺服类似，但一个关键区别在于，公式(4)保证时间步结束时的电机角度和速度满足约束，而PD伺服则使用当前电机角度和速度来决定如何驱动电机。因此，如果使用较大的增益，电机在仿真中能保持稳定，但在现实中会振荡。

为了消除执行器的模型差异，我们根据理想直流电机的动力学开发了一个执行器模型。给定PWM信号，电机的转矩为

$$\begin{aligned} \tau &= K_t I \\ I &= \frac{V_{\text{pwm}} - V_{\text{emf}}}{R} \qquad (5) \\ V_{\text{emf}} &= K_t \dot{q} \qquad (6) \end{aligned}$$

其中 $I$ 是电枢电流，$K_t$ 是转矩常数或反电动势（EMF）常数，$V_{\text{pwm}}$ 是由PWM信号调制的供电电压，$V_{\text{emf}}$ 是反电动势电压，$R$ 是电枢电阻。参数 $K_t$ 和 $R$ 在执行器规格书中提供。

利用上述模型，我们观察到真实的 Minitaur 经常下沉到脚部或无法抬起脚，而相同的控制器在仿真中却工作正常。这是因为线性转矩-电流关系仅在理想电机上成立。实际上，转矩会随着电流增加而饱和。因此，我们构造了一个分段线性函数来表征这种非线性转矩-电流关系 [54]。在仿真中，一旦根据 PWM（式（5）和（6））计算出电流，我们就使用该分段函数查找对应的转矩。

在位置控制中，PWM 由 PD 伺服器控制。

$$V_{\text{pwm}} = V(k_p(\bar{q} - q_n) + k_d(\bar{\dot{q}} - \dot{q}_n)) \tag{7}$$

其中 $V$ 是电池电压。请注意，我们使用当前时间步的角度和速度。此外，在参考 Ghost Robotics 在微控制器上的 PD 实现后，我们设置目标速度 $\bar{\dot{q}} = 0$。

我们进行了一项实验来验证新的执行器模型。我们使用正弦曲线作为期望轨迹来驱动电机。借助该执行器模型，仿真轨迹与真实轨迹（图4）一致。

[1]对于难以拆卸的机器人，可以应用传统的系统辨识方法。

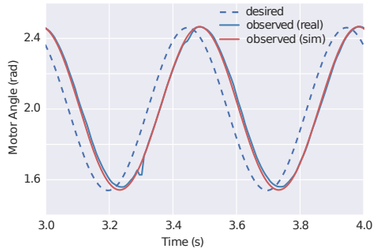



Fig. 4: Comparison of the simulated motor trajectory (red) with the ground truth (blue).

*b) Latency:* Latency is one of the main causes of instability for feedback control. It is the time delay between when a motor command is sent that causes the state of the robot to change and when the sensor measurement of this change is reported back to the controller. In Bullet, the motor commands take effect immediately and the sensors report back the state instantaneously. This instantaneous feedback makes the stability region of a feedback controller in simulation much larger than its implementation on hardware. For this reason, we often see a feedback policy learned in simulation starts to oscillate, diverge and ultimately fail in the real world.

To model latency, we keep a history of observations and their measurement time $\{(t_i, \mathbf{O}_i)_{i=0,1,\ldots,n-1}\}$, where $t_i = i\Delta t$ and $\Delta t$ is the time step. At the current step $n$, when the controller needs an observation, we search the history for two adjacent observations $\mathbf{O}_i$ and $\mathbf{O}_{i+1}$ where $t_i \leq n\Delta t - t_{\text{latency}} \leq t_{i+1}$ and linearly interpolate them.

To measure the latency on the physical system, we send a spike of PWM signal that lasts for one time step, which causes a small movement of the motor. We measure the time delay between when the spike is sent and when the resultant motor movement is reported. Note that we have two different latencies for the microcontroller and Nvidia Jetson TX2. The PD servo running on the microcontroller has a lower latency (3ms) while the locomotion controller executed on TX2 has a higher latency (typically 15-19ms). We use these measurements to set the correct latencies in simulation.

### *B. Learning Robust Controllers*

Robust control aims at achieving robust performance in the presence of model error. Thus, it is easier to transfer a robust controller to the real world even if the simulated dynamics are not identical to their real-world counterparts. In this paper, we experiment with three different ways to learn a robust controller: randomizing dynamic parameters, adding random perturbations and using a compact observation space.

Prior studies showed that randomizing the dynamic parameters during training can increase the robustness of the learned controller [39, 44]. At the beginning of each training episode, we randomly sample a set of physical parameters and use them in the simulation. The samples are drawn uniformly within

TABLE I: Randomized physical parameters and their ranges.

| parameter | lower bound | upper bound |
|---|---|---|
| mass | 80% | 120% |
| motor friction | 0Nm | 0.05Nm |
| inertia | 50% | 150% |
| motor strength | 80% | 120% |
| control step | 3ms | 20ms |
| latency | 0ms | 40ms |
| battery voltage | 14.0V | 16.8V |
| contact friction | 0.5 | 1.25 |
| IMU bias | -0.05 radian | 0.05 radian |
| IMU noise (std) | 0 radian | 0.05 radian |

certain ranges. The complete list of randomized parameters and their ranges of randomization is summarized in Table I.

Since dynamics randomization trades optimality for robustness [55], we choose the parameters and their ranges in Table I carefully to prevent learning overly conservative running gaits. Mass and motor friction are commonly randomized parameters [44, 39]. While we give them conservative ranges since we have measured them during system identification, we are less certain about inertia because it is estimated based on a uniform density assumption. Also some quantities can change over time. For example, the motor strength can vary due to wear and tear. The control step and latency can fluctuate because of the non real-time system. The battery voltage can change based on whether it is fully charged. For the reasons stated above, we choose to randomize these parameters and their ranges based on the actual measurements plus small safety margins. Performing accurate identification for contact parameters is challenging. While the Bullet physics library uses an LCP-based contact model and provides a number of tunable parameters, we choose to focus on the lateral friction and leave others in their default values. We randomly sample the friction coefficient between 0.5 and 1.25 because this is the typical range of friction coefficient between the Minitaur's rubber feet and various carpet floors [56]. Since the real IMU measurement often carries bias and noise, we also add a small amount of Gaussian noise to the simulated IMU readings.

Another popular method to improve the robustness of a learned controller is to add random perturbations [41]. During training, we add a perturbation force to the base of the simulated Minitaur every 200 steps of simulation (1.2s). The perturbation force lasts for ten steps (0.06s), has a random direction and a random magnitude ranging from 130N to 220N. The perturbations can knock the simulated Minitaur out of balance so that it needs to learn how to recover balance from different situations.

We find that the design of the observation space also plays an important role to narrow the reality gap. If the observation space is high dimensional, the learned policy can easily overfit the simulated environment, which makes it difficult to transfer to the real robots. In this paper, we use a compact observation space (Section IV-B) that leaves less room for overfitting to unimportant details of the simulation. A detailed analysis on the choice of observation space is presented in Section VI-B.

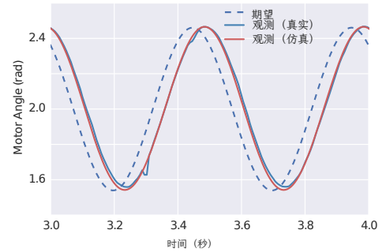



图4：模拟电机轨迹（红色）与真实轨迹（蓝色）的对比。

b) 延迟：延迟是导致反馈控制不稳定的主要原因之一。它是指从发送电机指令改变机器人状态到传感器测量值反馈给控制器之间的时间延迟。在Bullet中，电机指令立即生效，传感器也即时反馈状态。这种即时反馈使得仿真中反馈控制器的稳定区域远大于在硬件上的实现。因此，我们经常看到在仿真中学习的反馈策略在现实世界中开始振荡、发散并最终失败。

为了对延迟进行建模，我们保留观测值及其测量时间 $\{(t_i, O_i)_{i=0,1,\ldots,n-1}\}$ 的历史记录，其中 $t_i = i\Delta t$ 和 $\Delta t$ 是时间步长。在当前时间步 $n$，当控制器需要观测值时，我们在历史记录中搜索两个相邻的观测值 $O_i$ 和 $O_{i+1}$，其中 $t_i \leq n\Delta t - t_{\text{延迟}} \leq t_{i+1}$，然后对它们进行线性插值。

为了测量物理系统上的延迟，我们发送一个持续一个时间步的PWM信号尖峰，该信号会引起电机的微小运动。我们测量从发送尖峰到报告相应电机运动之间的时间延迟。请注意，微控制器和Nvidia Jetson TX2存在两种不同的延迟。运行在微控制器上的PD伺服延迟较低（3ms），而在TX2上执行的运动控制器延迟较高（通常为15-19ms）。我们利用这些测量结果在仿真中设置正确的延迟。

### B. 学习鲁棒控制器

鲁棒控制旨在在存在模型误差的情况下实现鲁棒性能。因此，即使模拟动力学与真实世界的动力学不完全相同，鲁棒控制器也更容易迁移到真实世界。在本文中，我们尝试了三种学习鲁棒控制器的不同方法：随机化动力学参数、添加随机扰动以及使用紧凑的观测空间。

先前的研究表明，在训练过程中随机化动态参数可以提高所学控制器的鲁棒性 [39, 44]。在每个训练回合开始时，我们随机采样一组物理参数并在仿真中使用它们。样本均匀地抽取自

表 I：随机化物理参数及其范围。

| 参数 | 下限 | 上限 |
|---|---|---|
| mass | 80% | 120% |
| 电机摩擦 | 0Nm | 0.05Nm |
| 惯量 | 50% | 150% |
| 电机强度 | 80% | 120% |
| 控制步长 | 3ms | 20ms |
| 延迟 | 0ms | 40ms |
| 电池电压 | 14.0V | 16.8V |
| 接触摩擦 | 0.5 | 1.25 |
| IMU偏置 | -0.05 弧度 | 0.05 弧度 |
| IMU噪声（标准差） | 0 弧度 | 0.05 弧度 |

某些范围。完整的随机化参数列表及其随机化范围总结在表 I 中。

由于动力学随机化以最优性换取鲁棒性 [55]，，我们仔细选择表 I 中的参数及其范围，以避免学习到过于保守的奔跑步态。质量和电机摩擦力是常见的随机化参数 [44, 39]。虽然我们根据系统辨识时的测量结果给它们设置了保守的范围，但对转动惯量则不那么确定，因为它是基于均匀密度假设估算出来的。此外，有些量会随时间变化。例如，电机强度会因磨损而改变；控制步长和延迟会因非实时系统而波动；电池电压会因是否充满电而变化。基于上述原因，我们选择在实际测量值的基础上加上较小的安全余量，来随机化这些参数及其范围。对接触参数进行精确辨识具有挑战性。虽然 Bullet 物理库使用基于 LCP 的接触模型并提供许多可调参数，但我们选择重点关注横向摩擦力，其余参数则保持默认值。我们随机采样摩擦系数在 0.5 到 1.25 之间，因为这是 Minitaur 的橡胶脚掌与各种地毯地面之间摩擦系数的典型范围 [56]。由于真实 IMU 测量通常带有偏置和噪声，我们还在模拟的 IMU 读数中添加了少量高斯噪声。

另一种提高学习控制器鲁棒性的常用方法是添加随机扰动 [41]。在训练过程中，我们每模拟 200 步（1.2 秒）就对模拟 Minitaur 的机体施加一次扰动力。该扰动力持续十步（0.06 秒），方向随机，大小在 130N 到 220N 之间随机。这些扰动可能使模拟 Minitaur 失去平衡，因此它需要学会如何从不同情况中恢复平衡。

我们发现，观测空间的设计对于缩小现实差距也起着重要作用。如果观测空间是高维的，学习到的策略很容易过拟合模拟环境，从而难以迁移到真实机器人。在本文中，我们使用紧凑的观测空间（第 IV-B 节），以减少对模拟中无关紧要细节的过拟合空间。关于观测空间选择的详细分析见第 VI-B 节。

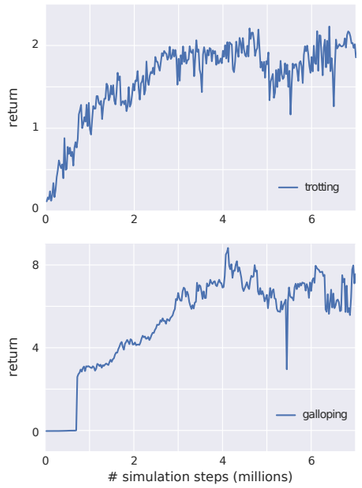



Fig. 5: The learning curves of trotting and galloping.

TABLE II: Parameters of learning algorithm for each task.

| Gait | Observation Dimension | Policy Net Size | Value Net Size | Learning Time (Hour) |
|---|---|---|---|---|
| Trotting | 4 | (125, 89) | (89, 55) | 4.35 |
| Galloping | 12 | (185, 95) | (95, 85) | 3.25 |

## VI. Evaluation and Discussion

We tested our system with two locomotion tasks: galloping and trotting. Please watch the accompanying video[2] for the learned locomotion gaits. We first learned the locomotion controllers in simulation. The simulated Minitaur environments are open-sourced in Bullet physics library[3]. We represented both the policy and the value functions as fully connected neural networks, each of which had two hidden layers. Their sizes were determined using hyperparameter search. At each iteration of policy update, we collected the simulated experience by running 25 roll-outs (up to 1000 steps each) in parallel. The training terminated after the maximum number of simulation steps (7 million) had been reached. The learning curves of trotting and galloping are shown in Figure 5, and the parameter settings are summarized in Table II. After the policies were learned, we deployed them on the real robot. The controllers worked directly in the real world without additional fine tuning on the physical system.

### A. Locomotion Tasks

In the first experiment, we let the system learn from scratch: We set the open loop component $\bar{\mathbf{a}}(t) = 0$ and gave the feedback component large output bounds: The bounds for swing are $[-0.5, 0.5]$ radians and the bounds for extension are $[\frac{\pi}{2} - 0.5, \frac{\pi}{2} + 0.5]$ radians. Initially, with the baseline simulation (no actuator model and latency), our system did not come up with an agile gait. Instead, a slow walking gait was learned in simulation. This might be caused by the default constraint-based actuator model in Bullet, which seems to be overdamped and limits the agility of the motion. Moreover, the real Minitaur fell to the ground immediately due to the reality gap. After we improved the simulation (Section V-A), an agile galloping gait emerged automatically. Galloping is the fastest gait for quadrupeds in nature. The running speed of the Minitaur reaches roughly 1.34 m/s (2.48 body lengths per second) in simulation and 1.18 m/s (2.18 body lengths per second) in the real world. We repeated the training with different hyperparameters and random seeds, and found that the majority of the solutions converged to galloping. We also observed a small number of other gaits, including trotting, pacing, and gaits that are uncommon among quadruped animals.

In the second experiment, we would like the Minitaur to learn trotting, an intermediate-speed running gait commonly adopted by horses (and many other quadrupeds) in which the diagonally opposite legs move together. While it is unclear how to use reward shaping to learn such a gait, we can directly control the learned gait by providing an open loop signal ($\bar{\mathbf{a}}(t)$ in eq. (3)). The signal consists of two sine curves.

$$\begin{aligned} \bar{s}(t) &= 0.3\sin(4\pi t) \\ \bar{e}(t) &= 0.35\sin(4\pi t) + 2 \end{aligned}$$

where $\bar{s}(t)$ is the signal for the swing and the $\bar{e}(t)$ is for the extension. One diagonal pair of legs shares the same curves and the other diagonal pair's are 180 degree out of phase. Note that while this open loop controller expresses the user's preference of the locomotion style, by itself, it cannot produce any forward movement in the real world: The Minitaur loses balance immediately and sits down on its rear legs. To maintain balance and keep the learned gait similar to the user-specified signal, we used small output bounds of the feedback controller: $[-0.25, 0.25]$ radians for both the swing and the extension degrees of freedom. After training with the improved simulator and random perturbations, the Minitaur is able to trot stably in simulation. However, when the policies were deployed on the robot, we had mixed results due to the reality gap: Some policies can transfer while others cannot. We further reduced the observation space to four dimensional, keeping only the IMU readings (the roll, pitch and the angular velocities of the base along these two axes) and retrained in simulation. This time, we observed stable, comparable movements in both simulation and on the real robot. The trotting speed is roughly 0.50 m/s (0.93 body lengths per second) in simulation and 0.60 m/s (1.11 body lengths per second) in the real world.

We compared the learned gaits with the handcrafted ones from Ghost Robotics [3]. Table III summarizes the speed and the energy consumption of these gaits on the real robots. While our learned gaits are as fast as the ones carefully tuned by experts, they consume significantly less power (35% and 23% reduction for galloping and trotting respectively).

[2]https://www.youtube.com/watch?v=lUZUr7jxoqM
[3]https://git.io/vp0V3

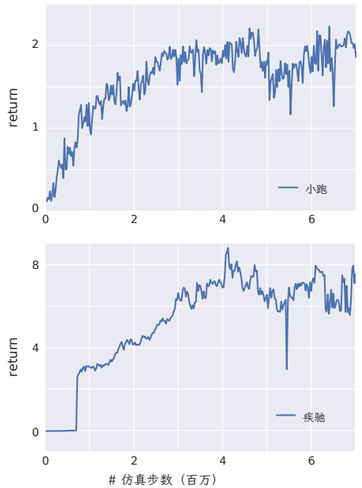



图5：小跑和疾驰的学习曲线。

表II：每项任务的学习算法参数。

| Gait | 观测维度 | 策略网络 Size | 价值网络 Size | 学习时间（小时） |
|---|---|---|---|---|
| 小跑 | 4 | (125, 89) | (89, 55) | 4.35 |
| 疾驰 | 12 | (185, 95) | (95, 85) | 3.25 |

## VI. 评估与讨论

我们使用两种运动任务测试了我们的系统：疾驰（galloping）和小跑（trotting）。请观看随附视频[2]，了解学习到的运动步态。我们首先在仿真中学习运动控制器。仿真的 Minitaur 环境已在 Bullet 物理库[3]中开源。我们将策略和价值函数均表示为全连接神经网络，每个网络都有两个隐藏层。其大小通过超参数搜索确定。在每次策略更新迭代中，我们并行运行 25 次 rollout（每次最多 1000 步）来收集仿真经验。当达到最大仿真步数（700 万）后，训练终止。疾驰和小跑的学习曲线如图 5 所示，参数设置汇总于表 II。策略学习完成后，我们将其部署到真实机器人上。控制器无需在物理系统上进行额外微调即可直接在真实世界中工作。

### A. 运动任务

在第一个实验中，我们让系统从零开始学习：我们设置开环组件 $\bar{a}(t) = 0$，并给予反馈组件较大的输出边界：摆动的边界为 $[-0.5, 0.5]$ 弧度，伸展的边界为 $[\frac{\pi}{2} - 0.5, \frac{\pi}{2} + 0.5]$ 弧度。最初，使用基线仿真（无执行器模型和延迟）中，我们的系统没有产生敏捷步态。相反，在仿真中学到的是缓慢的行走步态。这可能是由 Bullet 中默认的基于约束的执行器模型导致的，该模型似乎过阻尼并限制了运动的敏捷性。此外，由于现实差距，真实的 Minitaur 立即摔倒在地。在我们改进仿真（第 V-A 节）之后，一种敏捷的飞奔步态自动出现了。飞奔是自然界四足动物最快的步态。Minitaur 的奔跑速度在仿真中约为 1.34 米/秒（每秒 2.48 个身长），在现实世界中约为 1.18 米/秒（每秒 2.18 个身长）。我们使用不同的超参数和随机种子重复训练，发现大多数解都收敛到飞奔。我们还观察到少量其他步态，包括小跑、同侧步，以及一些在四足动物中不常见的步态。

在第二个实验中，我们希望 Minitaur 学会小跑——一种马（以及许多其他四足动物）常用的中等速度奔跑步态，在这类步态中，对角线方向的腿同步移动。虽然目前尚不清楚如何使用奖励塑形来学习这种步态，但我们可以通过提供一个开环信号（公式 (3) 中的 $\bar{a}(t)$）来直接控制学习到的步态。该信号由两条正弦曲线组成。

$$\begin{aligned} \bar{s}(t) &= 0.3\sin(4\pi t) \\ \bar{e}(t) &= 0.35\sin(4\pi t) + 2 \end{aligned}$$

其中 $\bar{s}(t)$ 是摆动信号，$\bar{e}(t)$ 是伸展信号。一对对角腿共享相同的曲线，另一对对角腿的曲线则与其相位相差 180 度。请注意，尽管这个开环控制器表达了用户对运动风格的偏好，但它本身无法在现实世界中产生任何前进运动：Minitaur 会立即失去平衡，并坐倒在后腿上。为了保持平衡，并使学习到的步态与用户指定的信号相似，我们使用了较小的反馈控制器输出边界：摆动和伸展两个自由度的输出边界均为 $[-0.25, 0.25]$ 弧度。在改进后的模拟器中加入随机扰动进行训练后，Minitaur 能够在仿真中稳定地小跑。然而，当这些策略被部署到机器人上时，由于现实差距，结果好坏参半：有些策略能够迁移，而有些则不能。我们进一步将观测空间缩减为四维，只保留 IMU 读数（横滚角、俯仰角以及基座绕这两个轴的角速度），并在仿真中重新训练。这一次，我们在仿真中和真实机器人上都观察到了稳定且相似的运动。小跑速度在仿真中约为 0.50 m/s（每秒 0.93 个身长），在现实世界中约为 0.60 m/s（每秒 1.11 个身长）。

我们将学习到的步态与来自 Ghost Robotics [3] 的手工设计步态进行了比较。表 III 总结了这些步态在真实机器人上的速度和能量消耗。虽然我们学习到的步态与专家精心调校的步态一样快，但它们的功耗显著更低（疾驰和小跑分别降低了35%和23%）。

### B. Narrowing the Reality Gap

As discussed in Section V-B, we narrow the reality gap by improving the simulation fidelity and learning robust controllers. Using trotting as a test case, we performed comprehensive evaluations to understand how each of these techniques, improving simulation, adding randomization and using compact observation space, can narrow the reality gap.

First, we defined a quantitative measure of the reality gap. One common choice is the success rate [43]: The percentage of the controllers that can balance in the real world for the entire episode (1000 steps, about 6 seconds). However, the binary outcome of success or failure does not capture the key characteristics of locomotion, such as running speed and energy consumption. For this reason, we adopt a continuous measure used by Koos et al. [7]. We compute the expected return from eq. (2), both in simulation and in the real experiments. Their difference measures the reality gap.

We observed a large variance of the controller performance in the real experiments due to the randomness of the learning algorithm, the stochastic nature of the real world, and the differences in robustness of the learned controllers. To increase the confidence of our conclusions, we performed a large number of learning trials and robot experiments before a conclusion is reached. In each of the experiments below, we trained 100 controllers with different hyperparameters and random seeds. We deployed the top three controllers on the Minitaur based on their returns in simulation. Each controller was run three times. The average return of these nine runs was reported as the expected return.

To examine how improving the simulation can narrow the reality gap, we trained three groups of controllers, all using the four-dimensional observations space. In the first group, the controllers were trained using the baseline simulation (no actuator model and latency handling). The second group used the baseline simulation with random perturbations during training. The last group was our method, which used the improved simulation and random perturbations. The top three controllers in each group all performed well in simulation. However, when they were deployed on the robot, the controllers in group I and II performed poorly. From the left and the center bars of Figure 6, we can clearly see a large reality gap (the difference between the blue and the red bars). In contrast, the controllers in group III performed comparably in simulation and in the real world (the right bars in Figure 6). This indicates that the improvement of the simulation is essential to our system. If the model discrepancy is too big, even a robust controller trained with random perturbations cannot overcome the reality gap.

TABLE III: Comparisons between learned and handcrafted gaits.

| Gait | Speed (m/s) | Avg. Mech. Power (watt) |
|---|---|---|
| Trotting (handcrafted) | 0.56 | 92.72 |
| Trotting (learned) | **0.60** | **71.78** |
| Galloping (handcrafted) | **1.21** | 290.00 |
| Galloping (learned) | 1.18 | **188.79** |

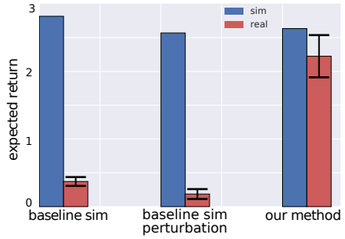



Fig. 6: Controller performance in simulation (blue) and on the robot (red). From left to right, the controllers are trained using baseline simulation, using baseline simulation with random perturbations, and using improved simulation with random perturbations. Error bars indicate one standard error.

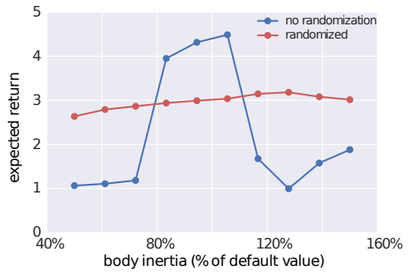



Fig. 7: Performance comparison of controllers that are trained with (red) and without (blue) randomization and tested with different body inertia.

We also found that both accurate actuator model and latency simulation are important. Without either of them, the learned controllers do not work on the real robot.

Next, we evaluated the impact of randomizing physical parameters and adding random perturbations on the robustness and transferability of controllers. Experiments showed that these two approaches had similar effects. In fact, as pointed out in the prior work [41], uncertainties of the physical parameters can be viewed as extra forces/torques applied to the system. For this reason, we grouped their results and reported them under the common name of "randomization". We first evaluated the trade-off between robustness and optimality using randomization. Given a trained controller, we computed its expected return in a set of different test environments in simulation: For each parameter in Table I, we evenly sampled 10 values within its range while keeping other parameters unchanged. We used these samples to construct different test environments. We computed the return of the controller in each test environment and observed how the performance changed across environments. For example, we tested two controllers, trained with or without randomization, in the environments that the inertia of the robot ranges from $50\%$ to $150\%$ of its default value. Figure 7 shows that the controller trained with randomization (red) performs similarly across


### B. Narrowing the Reality Gap

As discussed in Section V-B, we narrow the reality gap by improving the simulation fidelity and learning robust controllers. Using trotting as a test case, we performed comprehensive evaluations to understand how each of these techniques, improving simulation, adding randomization and using compact observation space, can narrow the reality gap.

First, we defined a quantitative measure of the reality gap. One common choice is the success rate [43]: The percentage of the controllers that can balance in the real world for the entire episode (1000 steps, about 6 seconds). However, the binary outcome of success or failure does not capture the key characteristics of locomotion, such as running speed and energy consumption. For this reason, we adopt a continuous measure used by Koos et al. [7]. We compute the expected return from eq. (2), both in simulation and in the real experiments. Their difference measures the reality gap.

We observed a large variance of the controller performance in the real experiments due to the randomness of the learning algorithm, the stochastic nature of the real world, and the differences in robustness of the learned controllers. To increase the confidence of our conclusions, we performed a large number of learning trials and robot experiments before a conclusion is reached. In each of the experiments below, we trained 100 controllers with different hyperparameters and random seeds. We deployed the top three controllers on the Minitaur based on their returns in simulation. Each controller was run three times. The average return of these nine runs was reported as the expected return.

To examine how improving the simulation can narrow the reality gap, we trained three groups of controllers, all using the four-dimensional observations space. In the first group, the controllers were trained using the baseline simulation (no actuator model and latency handling). The second group used the baseline simulation with random perturbations during training. The last group was our method, which used the improved simulation and random perturbations. The top three controllers in each group all performed well in simulation. However, when they were deployed on the robot, the controllers in group I and II performed poorly. From the left and the center bars of Figure 6, we can clearly see a large reality gap (the difference between the blue and the red bars). In contrast, the controllers in group III performed comparably in simulation and in the real world (the right bars in Figure 6). This indicates that the improvement of the simulation is essential to our system. If the model discrepancy is too big, even a robust controller trained with random perturbations cannot overcome the reality gap.

TABLE III: Comparisons between learned and handcrafted gaits.

| Gait | Speed (m/s) | Avg. Mech. Power (watt) |
|---|---|---|
| Trotting (handcrafted) | 0.56 | 92.72 |
| Trotting (learned) | **0.60** | **71.78** |
| Galloping (handcrafted) | **1.21** | 290.00 |
| Galloping (learned) | 1.18 | **188.79** |

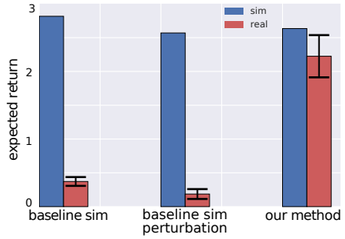



Fig. 6: Controller performance in simulation (blue) and on the robot (red). From left to right, the controllers are trained using baseline simulation, using baseline simulation with random perturbations, and using improved simulation with random perturbations. Error bars indicate one standard error.

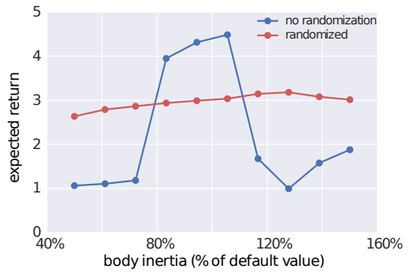



Fig. 7: Performance comparison of controllers that are trained with (red) and without (blue) randomization and tested with different body inertia.

We also found that both accurate actuator model and latency simulation are important. Without either of them, the learned controllers do not work on the real robot.

Next, we evaluated the impact of randomizing physical parameters and adding random perturbations on the robustness and transferability of controllers. Experiments showed that these two approaches had similar effects. In fact, as pointed out in the prior work [41], uncertainties of the physical parameters can be viewed as extra forces/torques applied to the system. For this reason, we grouped their results and reported them under the common name of "randomization". We first evaluated the trade-off between robustness and optimality using randomization. Given a trained controller, we computed its expected return in a set of different test environments in simulation: For each parameter in Table I, we evenly sampled 10 values within its range while keeping other parameters unchanged. We used these samples to construct different test environments. We computed the return of the controller in each test environment and observed how the performance changed across environments. For example, we tested two controllers, trained with or without randomization, in the environments that the inertia of the robot ranges from $50\%$ to $150\%$ of its default value. Figure 7 shows that the controller trained with randomization (red) performs similarly across

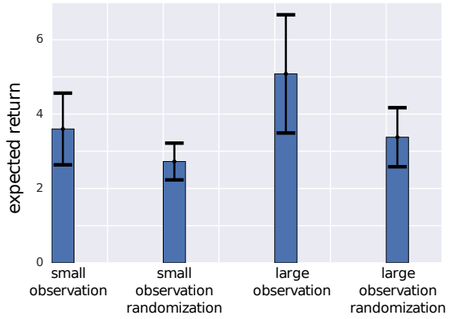



Fig. 8: Performance of controllers when they are tested in different simulation environments. Error bars indicate one standard deviation.

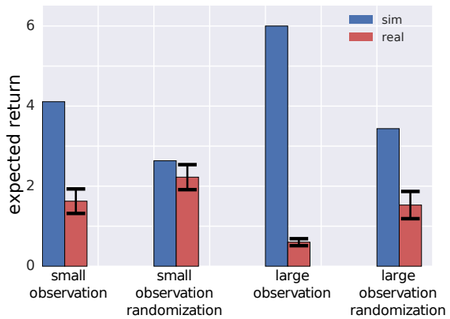



Fig. 9: Comparison of controllers trained with different observation spaces and randomization. The blue and red bars are the performance in simulation and in the real world respectively. Error bars indicate one standard error.

all these environments. In contrast, the performance of the controller trained without randomization can drop significantly if the actual inertia is different from what it was trained for. However, it has a higher peak performance.

We aggregated these results and calculated the mean and the standard deviation of the returns across all the test environments. It is obvious in Figure 8 that regardless of the choice of observation space, the returns of the controllers trained using randomization have a lower mean and a lower standard deviation (Refer to the first two bars in Figure 8 for the case of small observation space, and last two bars for the case of large observation space). Lower mean indicates suboptimal performance. The controllers learn to behave conservatively in the stochastic environments. Lower standard deviation indicates robustness. The performance varies less even when the controllers are tested in different environments. These results demonstrate a clear trade-off between robustness and optimality when randomization is used. Since randomization is not a free meal, we should use it only when necessary.

We then evaluated the impact of randomization on the transferability of controllers. Figure 9 shows that regardless of the choice of observations, randomization helps to narrow the reality gap. When combining it with the small observation space, we achieved the best results in the real world: All nine runs of the top three controllers in this group can trot more than three meters and balance for the entire episode.

Lastly, we examined the relation between the reality gap and the choice of observations. In this experiment, we trained controllers with two different observation space. The small space (4D) only consists of the IMU information: the roll, pitch, and the angular velocities of the base along these two axes. The large space (12D) consists of both the IMU information and the motor angles. We found that in simulation, when the observation space is large, the performance of the learned controllers is higher (Compare the blue bars between 1st and 3rd, and between 2nd and 4th in Figure 9). We suspect that the tasks become easier to learn because the policy can take advantage of the more information in the large observation space. However, when these policies are deployed on the robot, the result is the opposite. Controllers with the large observation space perform worse in the real world and the reality gap is wider (compare the red bars between 1st and 3rd, and between 2nd and 4th in Figure 9). We believe that this is caused by the mismatch of the observation distributions between training in simulation and testing on the physical system. When the space is large, the observations encountered in training become relatively sparse. Encountering similar observations in the real world is less likely, which can cause the robot to fall. In contrast, when the observation space is small, the observation distributions in training and testing are more likely to overlap, which narrows the reality gap.

## VII. Conclusion

We have shown that deep RL can be applied to learn agile locomotion automatically for robots. We have presented a complete system that applies deep RL in simulated environments, which can learn from scratch or can allow users to guide the learning process. With an accurate physical model and robust controllers, we have successfully deployed the controllers learned in simulation on the real robots. Using this system, we are able to develop two agile locomotion gaits, trotting and galloping, for a quadruped robot.

The focus of our paper is on learning transferable locomotion policies. For this purpose, we used a simple reward function and a simple environment: maximizing the running speed on a flat ground. In real-world scenarios, robots need to see the environment, adjust its speed and turn agilely to navigate the complex physical world. This points us to two interesting avenues for future work. First, we would like to learn locomotion policies that can dynamically change running speed and direction. Second, it would be interesting to extend this work to handle complex terrain structures by integrating vision as part of sensory input.

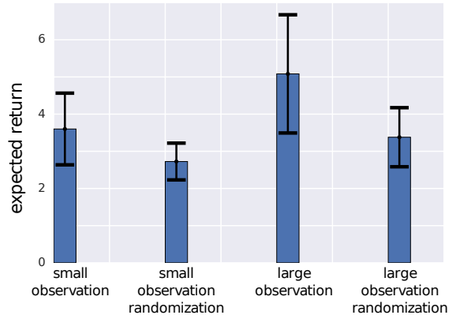




Fig. 8: Performance of controllers when they are tested in different simulation environments. Error bars indicate one standard deviation.

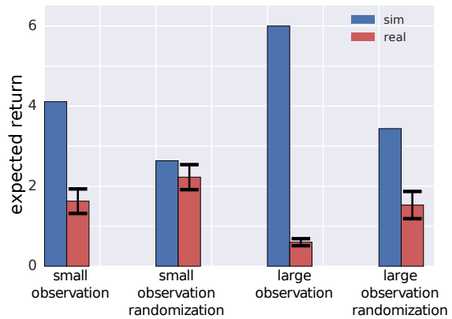



Fig. 9: Comparison of controllers trained with different observation spaces and randomization. The blue and red bars are the performance in simulation and in the real world respectively. Error bars indicate one standard error.

all these environments. In contrast, the performance of the controller trained without randomization can drop significantly if the actual inertia is different from what it was trained for. However, it has a higher peak performance.

We aggregated these results and calculated the mean and the standard deviation of the returns across all the test environments. It is obvious in Figure 8 that regardless of the choice of observation space, the returns of the controllers trained using randomization have a lower mean and a lower standard deviation (Refer to the first two bars in Figure 8 for the case of small observation space, and last two bars for the case of large observation space). Lower mean indicates suboptimal performance. The controllers learn to behave conservatively in the stochastic environments. Lower standard deviation indicates robustness. The performance varies less even when the controllers are tested in different environments. These results demonstrate a clear trade-off between robustness and optimality when randomization is used. Since randomization is not a free meal, we should use it only when necessary.

We then evaluated the impact of randomization on the transferability of controllers. Figure 9 shows that regardless of the choice of observations, randomization helps to narrow the reality gap. When combining it with the small observation space, we achieved the best results in the real world: All nine runs of the top three controllers in this group can trot more than three meters and balance for the entire episode.

Lastly, we examined the relation between the reality gap and the choice of observations. In this experiment, we trained controllers with two different observation space. The small space (4D) only consists of the IMU information: the roll, pitch, and the angular velocities of the base along these two axes. The large space (12D) consists of both the IMU information and the motor angles. We found that in simulation, when the observation space is large, the performance of the learned controllers is higher (Compare the blue bars between 1st and 3rd, and between 2nd and 4th in Figure 9). We suspect that the tasks become easier to learn because the policy can take advantage of the more information in the large observation space. However, when these policies are deployed on the robot, the result is the opposite. Controllers with the large observation space perform worse in the real world and the reality gap is wider (compare the red bars between 1st and 3rd, and between 2nd and 4th in Figure 9). We believe that this is caused by the mismatch of the observation distributions between training in simulation and testing on the physical system. When the space is large, the observations encountered in training become relatively sparse. Encountering similar observations in the real world is less likely, which can cause the robot to fall. In contrast, when the observation space is small, the observation distributions in training and testing are more likely to overlap, which narrows the reality gap.

## VII. Conclusion

We have shown that deep RL can be applied to learn agile locomotion automatically for robots. We have presented a complete system that applies deep RL in simulated environments, which can learn from scratch or can allow users to guide the learning process. With an accurate physical model and robust controllers, we have successfully deployed the controllers learned in simulation on the real robots. Using this system, we are able to develop two agile locomotion gaits, trotting and galloping, for a quadruped robot.

The focus of our paper is on learning transferable locomotion policies. For this purpose, we used a simple reward function and a simple environment: maximizing the running speed on a flat ground. In real-world scenarios, robots need to see the environment, adjust its speed and turn agilely to navigate the complex physical world. This points us to two interesting avenues for future work. First, we would like to learn locomotion policies that can dynamically change running speed and direction. Second, it would be interesting to extend this work to handle complex terrain structures by integrating vision as part of sensory input.

## References


[1] Marc H. Raibert. *Legged Robots That Balance*. Massachusetts Institute of Technology, Cambridge, MA, USA, 1986. ISBN 0-262-18117-7.

[2] Jerry Pratt and Gill Pratt. Intuitive control of a planar bipedal walking robot. In *Robotics and Automation, 1998. Proceedings. 1998 IEEE International Conference on*, volume 3, pages 2014–2021. IEEE, 1998.

[3] Avik De. Modular hopping and running via parallel composition, 2017.

[4] Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

[5] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017.

[6] Yan Duan, Xi Chen, Rein Houthooft, John Schulman, and Pieter Abbeel. Benchmarking deep reinforcement learning for continuous control. In *Proceedings of the 33rd International Conference on International Conference on Machine Learning - Volume 48*, ICML'16, pages 1329–1338. JMLR.org, 2016.

[7] Sylvain Koos, Jean-Baptiste Mouret, and Stéphane Doncieux. Crossing the reality gap in evolutionary robotics by promoting transferable controllers. In *Proceedings of the 12th annual conference on Genetic and evolutionary computation*, pages 119–126. ACM, 2010.

[8] Adrian Boeing and Thomas Bräunl. Leveraging multiple simulators for crossing the reality gap. In *Control Automation Robotics & Vision (ICARCV), 2012 12th International Conference on*, pages 1113–1119. IEEE, 2012.

[9] Sergey Levine, Peter Pastor, Alex Krizhevsky, Julian Ibarz, and Deirdre Quillen. Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection. *The International Journal of Robotics Research*.

[10] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354, 2017.

[11] Jens Kober and Jan Peters. Reinforcement learning in robotics: A survey. In *Reinforcement Learning*, pages 579–610. Springer, 2012.

[12] Krzysztof Choromanski, Atil Iscen, Vikas Sindhwani, Jie Tan, and Erwin Coumans. Optimizing simulations with noise-tolerant structured exploration. In *Robotics and Automation (ICRA), 2018 IEEE International Conference on*. IEEE, 2018.

[13] Antoine Cully, Jeff Clune, Danesh Tarapore, and Jean-Baptiste Mouret. Robots that can adapt like animals. *Nature*, 521(7553):503, 2015.

[14] Roberto Calandra, André Seyfarth, Jan Peters, and Marc Peter Deisenroth. Bayesian optimization for learning gaits under uncertainty. *Annals of Mathematics and Artificial Intelligence*, 76(1-2):5–23, 2016.

[15] Rika Antonova, Akshara Rai, and Christopher G. Atkeson. Deep kernels for optimizing locomotion controllers. In *1st Annual Conference on Robot Learning, CoRL*, pages 47–56, 2017.

[16] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. OpenAI Gym, 2016.

[17] Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, et al. DeepMind control suite. *arXiv preprint arXiv:1801.00690*, 2018.

[18] Danijar Hafner, James Davidson, and Vincent Vanhoucke. TensorFlow agents: Efficient batched reinforcement learning in TensorFlow. *arXiv preprint arXiv:1709.02878*, 2017.

[19] Hamid Benbrahim and Judy A Franklin. Biped dynamic walking using reinforcement learning. *Robotics and Autonomous Systems*, 22(3-4):283–302, 1997.

[20] Russ Tedrake, Teresa Weirui Zhang, and H Sebastian Seung. Stochastic policy gradient reinforcement learning on a simple 3d biped. In *Intelligent Robots and Systems, 2004.(IROS 2004). Proceedings. 2004 IEEE/RSJ International Conference on*, volume 3, pages 2849–2854. IEEE.

[21] Gen Endo, Jun Morimoto, Takamitsu Matsubara, Jun Nakanishi, and Gordon Cheng. Learning CPG sensory feedback with policy gradient for biped locomotion for a full-body humanoid. In *Proceedings of the 20th national conference on Artificial intelligence-Volume 3*, pages 1267–1273. AAAI Press, 2005.

[22] Nate Kohl and Peter Stone. Policy gradient reinforcement learning for fast quadrupedal locomotion. In *Proceedings of the IEEE International Conference on Robotics and Automation*, 2004.

[23] Masaki Ogino, Yutaka Katoh, Masahiro Aono, Minoru Asada, and Koh Hosoda. Reinforcement learning of humanoid rhythmic walking parameters based on visual information. *Advanced Robotics*, 18(7):677–697, 2004.

[24] Sébastien Gay, José Santos-Victor, and Auke Ijspeert. Learning robot gait stability using neural networks as sensory feedback function for central pattern generators. In *Intelligent Robots and Systems (IROS), 2013 IEEE/RSJ International Conference on*, pages 194–201. IEEE, 2013.

[25] Sergey Levine and Vladlen Koltun. Learning complex neural network policies with trajectory optimization. In *ICML '14: Proceedings of the 31st International Conference on Machine Learning*, 2014.

[26] Xue Bin Peng, Glen Berseth, and Michiel van de Panne. Dynamic terrain traversal skills using reinforcement learning. *ACM Trans. Graph.*, 34(4):80:1–80:11, 2015. ISSN 0730-0301. doi: 10.1145/2766910.

[27] Xue Bin Peng, Glen Berseth, and Michiel van de Panne.



## References

[1] Marc H. Raibert. *Legged Robots That Balance*. Massachusetts Institute of Technology, Cambridge, MA, USA, 1986. ISBN 0-262-18117-7.

[2] Jerry Pratt and Gill Pratt. Intuitive control of a planar bipedal walking robot. In *Robotics and Automation, 1998. Proceedings. 1998 IEEE International Conference on*, volume 3, pages 2014–2021. IEEE, 1998.

[3] Avik De. Modular hopping and running via parallel composition, 2017.

[4] Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

[5] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017.

[6] Yan Duan, Xi Chen, Rein Houthooft, John Schulman, and Pieter Abbeel. Benchmarking deep reinforcement learning for continuous control. In *Proceedings of the 33rd International Conference on International Conference on Machine Learning - Volume 48*, ICML'16, pages 1329–1338. JMLR.org, 2016.

[7] Sylvain Koos, Jean-Baptiste Mouret, and Stéphane Doncieux. Crossing the reality gap in evolutionary robotics by promoting transferable controllers. In *Proceedings of the 12th annual conference on Genetic and evolutionary computation*, pages 119–126. ACM, 2010.

[8] Adrian Boeing and Thomas Bräunl. Leveraging multiple simulators for crossing the reality gap. In *Control Automation Robotics & Vision (ICARCV), 2012 12th International Conference on*, pages 1113–1119. IEEE, 2012.

[9] Sergey Levine, Peter Pastor, Alex Krizhevsky, Julian Ibarz, and Deirdre Quillen. Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection. *The International Journal of Robotics Research*.

[10] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354, 2017.

[11] Jens Kober and Jan Peters. Reinforcement learning in robotics: A survey. In *Reinforcement Learning*, pages 579–610. Springer, 2012.

[12] Krzysztof Choromanski, Atil Iscen, Vikas Sindhwani, Jie Tan, and Erwin Coumans. Optimizing simulations with noise-tolerant structured exploration. In *Robotics and Automation (ICRA), 2018 IEEE International Conference on*. IEEE, 2018.

[13] Antoine Cully, Jeff Clune, Danesh Tarapore, and Jean-Baptiste Mouret. Robots that can adapt like animals. *Nature*, 521(7553):503, 2015.

[14] Roberto Calandra, André Seyfarth, Jan Peters, and Marc Peter Deisenroth. Bayesian optimization for learning gaits under uncertainty. *Annals of Mathematics and Artificial Intelligence*, 76(1-2):5–23, 2016.

[15] Rika Antonova, Akshara Rai, and Christopher G. Atkeson. Deep kernels for optimizing locomotion controllers. In *1st Annual Conference on Robot Learning, CoRL*, pages 47–56, 2017.

[16] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. OpenAI Gym, 2016.

[17] Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, et al. DeepMind control suite. *arXiv preprint arXiv:1801.00690*, 2018.

[18] Danijar Hafner, James Davidson, and Vincent Vanhoucke. TensorFlow agents: Efficient batched reinforcement learning in TensorFlow. *arXiv preprint arXiv:1709.02878*, 2017.

[19] Hamid Benbrahim and Judy A Franklin. Biped dynamic walking using reinforcement learning. *Robotics and Autonomous Systems*, 22(3-4):283–302, 1997.

[20] Russ Tedrake, Teresa Weirui Zhang, and H Sebastian Seung. Stochastic policy gradient reinforcement learning on a simple 3d biped. In *Intelligent Robots and Systems, 2004.(IROS 2004). Proceedings. 2004 IEEE/RSJ International Conference on*, volume 3, pages 2849–2854. IEEE.

[21] Gen Endo, Jun Morimoto, Takamitsu Matsubara, Jun Nakanishi, and Gordon Cheng. Learning CPG sensory feedback with policy gradient for biped locomotion for a full-body humanoid. In *Proceedings of the 20th national conference on Artificial intelligence-Volume 3*, pages 1267–1273. AAAI Press, 2005.

[22] Nate Kohl and Peter Stone. Policy gradient reinforcement learning for fast quadrupedal locomotion. In *Proceedings of the IEEE International Conference on Robotics and Automation*, 2004.

[23] Masaki Ogino, Yutaka Katoh, Masahiro Aono, Minoru Asada, and Koh Hosoda. Reinforcement learning of humanoid rhythmic walking parameters based on visual information. *Advanced Robotics*, 18(7):677–697, 2004.

[24] Sébastien Gay, José Santos-Victor, and Auke Ijspeert. Learning robot gait stability using neural networks as sensory feedback function for central pattern generators. In *Intelligent Robots and Systems (IROS), 2013 IEEE/RSJ International Conference on*, pages 194–201. IEEE, 2013.

[25] Sergey Levine and Vladlen Koltun. Learning complex neural network policies with trajectory optimization. In *ICML '14: Proceedings of the 31st International Conference on Machine Learning*, 2014.

[26] Xue Bin Peng, Glen Berseth, and Michiel van de Panne. Dynamic terrain traversal skills using reinforcement learning. *ACM Trans. Graph.*, 34(4):80:1–80:11, 2015. ISSN 0730-0301. doi: 10.1145/2766910.

[27] Xue Bin Peng, Glen Berseth, and Michiel van de Panne.

Terrain-adaptive locomotion skills using deep reinforcement learning. *ACM Transactions on Graphics (Proc. SIGGRAPH 2016)*, 35(4), 2016.

[28] Xue Bin Peng, Glen Berseth, KangKang Yin, and Michiel van de Panne. DeepLoco: Dynamic locomotion skills using hierarchical deep reinforcement learning. *ACM Transactions on Graphics (Proc. SIGGRAPH 2017)*, 36 (4), 2017.

[29] Nicolas Heess, Dhruva TB, Srinivasan Sriram, Jay Lemmon, Josh Merel, Greg Wayne, Yuval Tassa, Tom Erez, Ziyu Wang, S. M. Ali Eslami, Martin A. Riedmiller, and David Silver. Emergence of locomotion behaviours in rich environments. *CoRR*, abs/1707.02286, 2017.

[30] Arjun Sharma and Kris M. Kitani. Phase-parametric policies for reinforcement learning in cyclic environments. In *AAAI Conference on Artificial Intelligence*, Pittsburgh, PA, 2018.

[31] Michael Neunert, Thiago Boaventura, and Jonas Buchli. Why off-the-shelf physics simulators fail in evaluating feedback controller performance-a case study for quadrupedal robots. In *Advances in Cooperative Robotics*, pages 464–472. World Scientific, 2017.

[32] Shaojun Zhu, Andrew Kimmel, Kostas E. Bekris, and Abdeslam Boularias. Model identification via physics engines for improved policy search. *CoRR*, abs/1710.08893, 2017.

[33] Chen Li, Tingnan Zhang, and Daniel I Goldman. A terradynamics of legged locomotion on granular media. *Science*, 339(6126):1408–1412, 2013.

[34] Jie Tan, Zhaoming Xie, Byron Boots, and C. Karen Liu. Simulation-based design of dynamic controllers for humanoid balancing. In *Intelligent Robots and Systems (IROS), 2016 IEEE/RSJ International Conference on*, pages 2729–2736. IEEE, 2016.

[35] Rico Möckel, N. Perov Yura, Anh The Nguyen, Massimo Vespignani, Stephane Bonardi, Soha Pouya, Alexander Spröwitz, Jesse van den Kieboom, Frederic Wilhelm, and Auke Jan Ijspeert. Gait optimization for roombots modular robots - matching simulation and reality. In *Proceedings of the 2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 3265–3272, Tokyo, 2013. IEEE.

[36] Josh Bongard, Victor Zykov, and Hod Lipson. Resilient machines through continuous self-modeling. *Science*, 314:1118–21, 2006.

[37] Sehoon Ha and Katsu Yamane. Reducing hardware experiments for model learning and policy optimization. In *Robotics and Automation (ICRA), 2015 IEEE International Conference on*, pages 2620–2626. IEEE, 2015.

[38] Wenhao Yu, Jie Tan, C. Karen Liu, and Greg Turk. Preparing for the unknown: Learning a universal policy with online system identification. *CoRR*, abs/1702.02453, 2017.

[39] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. *arXiv preprint arXiv:1710.06537*, 2017.

[40] Nick Jakobi, Phil Husbands, and Inman Harvey. Noise and the reality gap: The use of simulation in evolutionary robotics. *Advances in artificial life*, pages 704–720, 1995.

[41] Lerrel Pinto, James Davidson, Rahul Sukthankar, and Abhinav Gupta. Robust adversarial reinforcement learning. *arXiv preprint arXiv:1703.02702*, 2017.

[42] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. *arXiv preprint arXiv:1703.06907*, 2017.

[43] Igor Mordatch, Kendall Lowrey, and Emanuel Todorov. Ensemble-CIO: Full-body dynamic motion planning that transfers to physical humanoids. In *Intelligent Robots and Systems (IROS), 2015 IEEE/RSJ International Conference on*, pages 5307–5314. IEEE, 2015.

[44] Aravind Rajeswaran, Sarvjeet Ghotra, Balaraman Ravindran, and Sergey Levine. EPOpt: Learning robust neural network policies using model ensembles. *arXiv preprint arXiv:1610.01283*, 2016.

[45] Andrei A. Rusu, Matej Vecerik, Thomas Rothörl, Nicolas Heess, Razvan Pascanu, and Raia Hadsell. Sim-to-real robot learning from pixels with progressive nets. *CoRR*, abs/1610.04286, 2016.

[46] Josiah Hanna and Peter Stone. Grounded action transformation for robot learning in simulation. In *Proceedings of the 31st AAAI Conference on Artificial Intelligence (AAAI)*, 2017.

[47] Eric Tzeng, Coline Devin, Judy Hoffman, Chelsea Finn, Xingchao Peng, Sergey Levine, Kate Saenko, and Trevor Darrell. Towards adapting deep visuomotor representations from simulated to real environments. *CoRR*, abs/1511.07111, 2015.

[48] Kuan Fang, Yunfei Bai, Stefan Hinterstoisser, and Mrinal Kalakrishnan. Multi-task domain adaptation for deep learning of instance grasping from simulation. *CoRR*, abs/1710.06422, 2017.

[49] Konstantinos Bousmalis, Alex Irpan, Paul Wohlhart, Yunfei Bai, Matthew Kelcey, Mrinal Kalakrishnan, Laura Downs, Julian Ibarz, Peter Pastor, Kurt Konolige, Sergey Levine, and Vincent Vanhoucke. Using simulation and domain adaptation to improve efficiency of deep robotic grasping. *CoRR*, abs/1709.07857, 2017.

[50] Gavin Kenneally, Avik De, and Daniel E Koditschek. Design principles for a family of direct-drive legged robots. *IEEE Robotics and Automation Letters*, 1(2):900–907, 2016.

[51] Erwin Coumans and Yunfei Bai. Pybullet, a python module for physics simulation in robotics, games and machine learning. http://pybullet.org, 2016–2017.

[52] Xue Bin Peng, Michiel van de Panne, and KangKang Yin. Learning locomotion skills using DeepRL: Does the choice of action space matter? In *Proc. ACM SIGGRAPH / Eurographics Symposium on Computer Animation*, 2017.

Terrain-adaptive locomotion skills using deep reinforcement learning. *ACM Transactions on Graphics (Proc. SIGGRAPH 2016)*, 35(4), 2016.

[28] Xue Bin Peng, Glen Berseth, KangKang Yin, and Michiel van de Panne. DeepLoco: Dynamic locomotion skills using hierarchical deep reinforcement learning. *ACM Transactions on Graphics (Proc. SIGGRAPH 2017)*, 36 (4), 2017.

[29] Nicolas Heess, Dhruva TB, Srinivasan Sriram, Jay Lemmon, Josh Merel, Greg Wayne, Yuval Tassa, Tom Erez, Ziyu Wang, S. M. Ali Eslami, Martin A. Riedmiller, and David Silver. Emergence of locomotion behaviours in rich environments. *CoRR*, abs/1707.02286, 2017.

[30] Arjun Sharma and Kris M. Kitani. Phase-parametric policies for reinforcement learning in cyclic environments. In *AAAI Conference on Artificial Intelligence*, Pittsburgh, PA, 2018.

[31] Michael Neunert, Thiago Boaventura, and Jonas Buchli. Why off-the-shelf physics simulators fail in evaluating feedback controller performance-a case study for quadrupedal robots. In *Advances in Cooperative Robotics*, pages 464–472. World Scientific, 2017.

[32] Shaojun Zhu, Andrew Kimmel, Kostas E. Bekris, and Abdeslam Boularias. Model identification via physics engines for improved policy search. *CoRR*, abs/1710.08893, 2017.

[33] Chen Li, Tingnan Zhang, and Daniel I Goldman. A terradynamics of legged locomotion on granular media. *Science*, 339(6126):1408–1412, 2013.

[34] Jie Tan, Zhaoming Xie, Byron Boots, and C. Karen Liu. Simulation-based design of dynamic controllers for humanoid balancing. In *Intelligent Robots and Systems (IROS), 2016 IEEE/RSJ International Conference on*, pages 2729–2736. IEEE, 2016.

[35] Rico Möckel, N. Perov Yura, Anh The Nguyen, Massimo Vespignani, Stephane Bonardi, Soha Pouya, Alexander Spröwitz, Jesse van den Kieboom, Frederic Wilhelm, and Auke Jan Ijspeert. Gait optimization for roombots modular robots - matching simulation and reality. In *Proceedings of the 2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 3265–3272, Tokyo, 2013. IEEE.

[36] Josh Bongard, Victor Zykov, and Hod Lipson. Resilient machines through continuous self-modeling. *Science*, 314:1118–21, 2006.

[37] Sehoon Ha and Katsu Yamane. Reducing hardware experiments for model learning and policy optimization. In *Robotics and Automation (ICRA), 2015 IEEE International Conference on*, pages 2620–2626. IEEE, 2015.

[38] Wenhao Yu, Jie Tan, C. Karen Liu, and Greg Turk. Preparing for the unknown: Learning a universal policy with online system identification. *CoRR*, abs/1702.02453, 2017.

[39] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. *arXiv preprint arXiv:1710.06537*, 2017.

[40] Nick Jakobi, Phil Husbands, and Inman Harvey. Noise and the reality gap: The use of simulation in evolutionary robotics. *Advances in artificial life*, pages 704–720, 1995.

[41] Lerrel Pinto, James Davidson, Rahul Sukthankar, and Abhinav Gupta. Robust adversarial reinforcement learning. *arXiv preprint arXiv:1703.02702*, 2017.

[42] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. *arXiv preprint arXiv:1703.06907*, 2017.

[43] Igor Mordatch, Kendall Lowrey, and Emanuel Todorov. Ensemble-CIO: Full-body dynamic motion planning that transfers to physical humanoids. In *Intelligent Robots and Systems (IROS), 2015 IEEE/RSJ International Conference on*, pages 5307–5314. IEEE, 2015.

[44] Aravind Rajeswaran, Sarvjeet Ghotra, Balaraman Ravindran, and Sergey Levine. EPOpt: Learning robust neural network policies using model ensembles. *arXiv preprint arXiv:1610.01283*, 2016.

[45] Andrei A. Rusu, Matej Vecerik, Thomas Rothörl, Nicolas Heess, Razvan Pascanu, and Raia Hadsell. Sim-to-real robot learning from pixels with progressive nets. *CoRR*, abs/1610.04286, 2016.

[46] Josiah Hanna and Peter Stone. Grounded action transformation for robot learning in simulation. In *Proceedings of the 31st AAAI Conference on Artificial Intelligence (AAAI)*, 2017.

[47] Eric Tzeng, Coline Devin, Judy Hoffman, Chelsea Finn, Xingchao Peng, Sergey Levine, Kate Saenko, and Trevor Darrell. Towards adapting deep visuomotor representations from simulated to real environments. *CoRR*, abs/1511.07111, 2015.

[48] Kuan Fang, Yunfei Bai, Stefan Hinterstoisser, and Mrinal Kalakrishnan. Multi-task domain adaptation for deep learning of instance grasping from simulation. *CoRR*, abs/1710.06422, 2017.

[49] Konstantinos Bousmalis, Alex Irpan, Paul Wohlhart, Yunfei Bai, Matthew Kelcey, Mrinal Kalakrishnan, Laura Downs, Julian Ibarz, Peter Pastor, Kurt Konolige, Sergey Levine, and Vincent Vanhoucke. Using simulation and domain adaptation to improve efficiency of deep robotic grasping. *CoRR*, abs/1709.07857, 2017.

[50] Gavin Kenneally, Avik De, and Daniel E Koditschek. Design principles for a family of direct-drive legged robots. *IEEE Robotics and Automation Letters*, 1(2):900–907, 2016.

[51] Erwin Coumans and Yunfei Bai. Pybullet, a python module for physics simulation in robotics, games and machine learning. http://pybullet.org, 2016–2017.

[52] Xue Bin Peng, Michiel van de Panne, and KangKang Yin. Learning locomotion skills using DeepRL: Does the choice of action space matter? In *Proc. ACM SIGGRAPH / Eurographics Symposium on Computer Animation*, 2017.

[53] URDF - ROS wiki. http://wiki.ros.org/urdf.
[54] Avik De and Daniel E. Koditschek. The Penn Jerboa: A platform for exploring parallel composition of templates. *CoRR*, abs/1502.05347, 2015.
[55] Jingru Luo and Kris Hauser. Robust trajectory optimization under frictional contact with iterative learning. *Autonomous Robots*, 41(6):1447–1461, 2017.
[56] Ikuko Hirai and Toshihiro Gunji. Slipperiness and coefficient of friction on the carpets. *Sen'i Kikai Gakkaishi (Journal of the Textile Machinery Society of Japan)*, 53: T140–T146, 2000.

[53] URDF - ROS wiki. http://wiki.ros.org/urdf.
[54] Avik De and Daniel E. Koditschek. The Penn Jerboa: A platform for exploring parallel composition of templates. *CoRR*, abs/1502.05347, 2015.
[55] Jingru Luo and Kris Hauser. Robust trajectory optimization under frictional contact with iterative learning. *Autonomous Robots*, 41(6):1447–1461, 2017.
[56] Ikuko Hirai and Toshihiro Gunji. Slipperiness and coefficient of friction on the carpets. *Sen'i Kikai Gakkaishi (Journal of the Textile Machinery Society of Japan)*, 53: T140–T146, 2000.